

The Bilinear Complexity and Practical Algorithms for Matrix Multiplication

A. V. Smirnov

*Russian Federal Center of Forensic Science, Department of Justice,
Khokhlovskii pereul. 13-2, Moscow, 109028 Russia
e-mail: S.Alexey.V@rambler.ru*

Received March 19, 2013; in final form, June 4, 2013

Abstract—A method for deriving bilinear algorithms for matrix multiplication is proposed. New estimates for the bilinear complexity of a number of problems of the exact and approximate multiplication of rectangular matrices are obtained. In particular, the estimate for the boundary rank of multiplying 3×3 matrices is improved and a practical algorithm for the exact multiplication of square $n \times n$ matrices is proposed. The asymptotic arithmetic complexity of this algorithm is $O(n^{2.7743})$.

DOI: 10.1134/S0965542513120129

Keywords: bilinear complexity, rank of the matrix multiplication problem, boundary rank, algorithms for exact and approximate matrix multiplication, least-squares method, objective function.

More than 40 years passed from 1969, the year when an unexpected result of Volker Strassen directed attention of researchers to the problem of fast multiplication of large matrices (see [1]). Despite the efforts of numerous mathematicians and a number of prominent achievements, this problem in its general form is not yet solved in 2013. The conjectured asymptotic complexity estimate $O(n^{2+\varepsilon})$, where $\varepsilon > 0$ is an arbitrary positive scalar, is not proved as well. The algorithm proposed in [2] yields the exponent $p = 2.376$, while the best available theoretical estimate for p , based on some modifications of this algorithm, is $p = 2.373$. Such a negligible progress makes it possible to speak of the Coppersmith–Winograd barrier in the complexity theory. Another important point is that this algorithm is unpractical due to the astronomically large constant in the complexity estimate. Publications dealing with computer implementations of practical algorithms for fast multiplication consider normally only the algorithms with large exponents. For instance, Strassen’s algorithm with $p = 2.8074$ is used in [3], while the paper [4] uses the algorithm with $p = 2.7760$ proposed in [5]. It was found in [5] that the latter algorithm is not faster than Strassen’s algorithm as long as the order n does not exceed 4640. Here, we also consider bilinear algorithms for approximate multiplication of matrices with the product of dimensions at most 6^3 . Therefore, our reference point is the algorithm for approximate multiplication with $p = 2.7713$ proposed in [6]. No appreciable progress was recently made in the research, and the interest of mathematicians in this difficult problem begins to abate. On the other hand, the development of algorithms with the lowest possible complexity estimate for matrix multiplication was designated as the fourth-most important problem in the list of seven basic problems of computational linear algebra compiled in [7]. A detailed survey of the material published before 1988 is given in [8]. The reader who is a novice at this problem will find in [8] the definitions of bilinear algorithms and other terms used in this paper, as well as the proofs of certain properties of the bilinear algorithms that are briefly described below. Presently, the most comprehensive survey of the literature on this subject is [9].

The aim of this paper is to estimate the bilinear complexity of the multiplication problem and to look for faster practical algorithms for matrix multiplication. The multiplication problem can be described by a system of nonlinear equations. Below, we discuss our attempt at solving this system for small dimensions of matrix factors.

A practical algorithm is understood here as an algorithm for fast matrix multiplication that can be implemented on modern computers and has the following property: at least for some matrices, the number of arithmetic operations required for their multiplication is less than that for the classical multiplication method. Defined in this way, the practicality separates the algorithms examined in this paper from purely theoretical constructions of astronomical dimensions. It is a necessary condition for the practical utility of an algorithm, though not a sufficient one. However, in this paper, we do not consider all the aspects of the implementation of the practically useful algorithms.

All the reported calculations were performed on an ordinary personal computer with an accuracy up to 15 decimal digits.

Consider the problem of multiplying an $n_1 \times n_2$ matrix by an $n_2 \times n_3$ matrix. The minimum number of active multiplications in a noncommutative algorithm for this problem is denoted by $R = R(n_1, n_2, n_3)$ and is called the *bilinear complexity* or the *rank of the matrix multiplication problem*. It is known that $R < n_1 n_2 n_3$ if all the three dimensions are greater than one. This inequality, combined with the fact that the algorithm under consideration works for any noncommutative ring, including the matrix ring, makes it possible to design algorithms for fast multiplication of large matrices (see [1]).

The algorithms for fast multiplication of large matrices analyzed in this paper are determined by certain algorithms for multiplying two matrices of small specific dimensions. These algorithms for calculating $G = AB$ use a small number T of active multiplications ($R \leq T < n_1 n_2 n_3$). They can be described in terms of the entries (coefficients) of the matrices G , A , and B :

$$\begin{aligned}\gamma_{i_1 i_2}^t, \quad i_1 = 1, 2, \dots, n_1, \quad i_2 = 1, 2, \dots, n_3, \quad t = 1, 2, \dots, T, \\ \alpha_{j_1 j_2}^t, \quad j_1 = 1, 2, \dots, n_1, \quad j_2 = 1, 2, \dots, n_2, \quad t = 1, 2, \dots, T, \\ \beta_{k_1 k_2}^t, \quad k_1 = 1, 2, \dots, n_2, \quad k_2 = 1, 2, \dots, n_3, \quad t = 1, 2, \dots, T.\end{aligned}$$

These entries satisfy the system of equations

$$\sum_{t=1}^T \gamma_{i_1 i_2}^t \alpha_{j_1 j_2}^t \beta_{k_1 k_2}^t = L_{i_1 j_1 j_2 k_1 k_2}, \quad (1)$$

where

$$\begin{aligned}L_{i_1 j_1 j_2 k_1 k_2} &= \delta_{i_1 j_1} \delta_{j_2 k_1} \delta_{i_2 k_2}, \\ \delta_{ij} &= \begin{cases} 0, & i \neq j, \\ 1, & i = j \end{cases}\end{aligned}$$

(see [10]).

The minimum $T = R$ for which system (1) is solvable is the rank of the matrix multiplication problem. It is easy to show that $R = R(n_1, n_2, n_3)$ is a symmetric function; consequently, we can estimate the bilinear complexity of matrix multiplication only for problems with the sizes (n_1, n_2, n_3) arranged in nondecreasing order. To specify a solution to problem (1), we indicate not only the dimensions of matrices but also the number of the summands on the left-hand side of (1), separated by a semicolon. This number is called the *length of a solution* and may not be equal to the rank of the problem. Bilinear algorithms for multiplying two matrices of given sizes are completely determined by the corresponding solutions to system (1). Therefore, in what follows, we call these algorithms *solutions* in order to distinguish them from the algorithms for multiplying large matrices that originate from the former ones.

The bilinear complexity of problem (2, 2, 2) is known to be seven (see [11]), while that of problem (2, 2, 3) is 11 (see [12]). Solutions of the type (3, 3, 3; 23) for multiplying 3×3 matrices were proposed in [13, 14]. For these solutions, the total number of nonzero coefficients $\gamma_{i_1 i_2}^t$, $\alpha_{j_1 j_2}^t$, and $\beta_{k_1 k_2}^t$ is equal to 153. A solution of the type (4, 4, 4; 49) for 4×4 matrices can be constructed from the solution (2, 2, 2; 7). For 5×5 matrices, a solution of the type (5, 5, 5; 100) was proposed in [15].

It is possible that, in system (1), the sum S_1 of the squared differences between the left- and right-hand sides can be arbitrarily close to zero, and, nevertheless, the system is unsolvable. The minimum number of the summands in (1) for which such a situation occurs is called the *bilinear complexity of the problem of approximate matrix multiplication* or the *boundary rank*. It is denoted by $r = r(n_1, n_2, n_3)$. A collection of functions $\gamma_{i_1 i_2}^t(x)$, $\alpha_{j_1 j_2}^t(x)$, and $\beta_{k_1 k_2}^t(x)$ of the real variable such that $S_1(x) = O(x)$ is called a functional solution to the problem of approximate matrix multiplication.

It was shown in [16] that $r(2, 2, 3) \leq 10$. A functional solution of the type (3, 3, 3; 21) such that all the constituent functions are Laurent monomials was found in [6]. It was also proved in [6] that $r(4, 4, 4) \leq 47$.

It can be shown that, from arbitrary bilinear solutions of the types $(n_1, n_2, n_3; r_1)$ and $(m_1, m_2, m_3; r_2)$, one can construct a bilinear solution of the type $(n_1 m_1, n_2 m_2, n_3 m_3; r_1 r_2)$. This solution is called the *composition* or the *product of the original solutions*. From every solution to the problem of multiplying rectangular matrices, one can easily derive two related solutions of the same length by cyclically permuting the dimensions. The product of all the three rectangular solutions yields a solution for square matrices of the

order $n = n_1 n_2 n_3$. Such a symmetrization procedure is also possible for different rectangular solutions, for instance, solutions of the types $(2, 2, 3; r_1)$ and $(3, 3, 2; r_2)$.

The combination of solutions of the types $(n_1, n_2, k; r_1)$ and $(n_1, n_2, m; r_2)$ obtained by the column juxtaposition of the second factors yields obviously a solution of the type $(n_1, n_2, k + m; r_1 + r_2)$. We call this solution the *sum of the original solutions*. Suppose that we have a solution of the type $(n_1, 2, k; r_1)$ to the problem of partial matrix multiplication (see [6]) in which the entry b_{1k} is assigned the zero value as well as a solution of the type $(n_1, 2, m; r_2)$ for the problem with the zero entry b_{21} . It is easy to see that these solutions can be combined by superimposing the columns containing the zero entries, which gives a solution of the type $(n_1, 2, k + m - 1; r_1 + r_2)$ for the ordinary problem of matrix multiplication. In this paper, we use such reductions of asymmetric problems to two smaller problems in order to substantially improve the estimates for the boundary ranks.

A bilinear algorithm for fast matrix multiplication involves a representation of large matrices as low-dimensional matrices whose entries are submatrices and the recursive use of solutions of a low dimension and small length for multiplying the entries of low-dimensional matrices.

Suppose that an algorithm of this type for multiplying square matrices is based on a single solution for rectangular matrices. It can be shown (see [8]) that, in the asymptotic estimate $O(n^p)$ of the arithmetic complexity of this algorithm, the exponent p is given by the formula

$$p = 3 \log_{n_1 n_2 n_3}(r_1). \quad (2)$$

The properties of solutions discussed above and the operations defined on these solutions are all that we need to present our results.

Observe that the number of equations in system (1) is not identical to the number of unknowns. Replacing each paired index in this system by a single one, we rewrite the objective function in the form

$$S_1 = \sum_{i=1}^{N_1} \sum_{j=1}^{N_2} \sum_{k=1}^{N_3} \left(\sum_{t=1}^T \gamma_i^t \alpha_j^t \beta_k^t - L_{ijk} \right)^2. \quad (3)$$

Here, $N_1 = n_1 n_3$, $N_2 = n_1 n_2$, and $N_3 = n_2 n_3$.

An application of the least-squares method (LSM; see [17]) to this function yields a system of fifth degree equations. It is easy to see that this system can be split into three linear (matrix) subsystems, each for its own group of the unknown coefficients. By solving any of these subsystems, we obtain a solution to the partial problem of minimizing the objective function with respect to the corresponding group of unknowns under the given values of the unknowns in two other groups. The successive iterative solution of these matrix subsystems is a fairly rapid process, which does not increase the objective function and, under certain conditions, converges to a global solution.

Computational experiments show that, for all tested problems of full matrix multiplication, the local extrema and lower bounds of objective function (3) are integer-valued. In particular, if a value $S_1 < 1$ is obtained, then, in all probability, S_1 will tend to zero in the process. The only obstacle to this convergence is the loss in the accuracy of computations caused by the growth (in modulus) of the solutions to the partial multiplication problems. Suppose that an exact solution of the required length was not found. Then the collection of coefficients for which the last inequality in the process is satisfied is called a *numerical solution to the problem of approximate full matrix multiplication*. For partial multiplication problems, the lower bounds of the objective function may not be integer-valued. No limit points of the objective function in the interval $(0, 3/4)$ were found. Hypothetically, this fact makes it possible to determine the existence of an approximate solution.

The minimization of S_1 in the way indicated above and the subsequent use of results obtained causes the following four problems:

1. In the course of iteration, the unknown coefficients can grow indefinitely (in modulus). This is true even in the case where the exact solution is known to exist. Such a growth rapidly decreases the accuracy of solving systems of equations in the LSM.
2. For the problem of exact matrix multiplication, solutions with a small number of different values are more conclusive and practically useful. For instance, all the desired coefficients may belong to the set $\{-1, 0, 1\}$. However, the iterative process by itself does not lead to a solution of this kind. In what follows, a solution to the problem of exact multiplication having at most five different values of the coefficients is called a *discrete solution*.

3. The only thing made possible by the existence of a numerical solution to the problem of approximate multiplication is the conjecture that an approximate functional solution does exist. To justify an estimate for the boundary rank, this functional solution must be constructed.

4. Even in the cases where a solution is known to exist, the process usually begins to “stick” at local positive integer-valued extrema when the sizes of matrices grow.

Now, we introduce the objective function

$$S_2 = S_1 + \varepsilon \sum_{t=1}^T \left(\sum_{i=1}^{N_1} (\gamma_i^t - \tilde{\gamma}_i^t)^2 + \sum_{j=1}^{N_2} (\alpha_j^t - \tilde{\alpha}_j^t)^2 + \sum_{k=1}^{N_3} (\beta_k^t - \tilde{\beta}_k^t)^2 \right). \quad (4)$$

Here, ε is a nonnegative number, while $\tilde{\gamma}_i^t$, $\tilde{\alpha}_j^t$, and $\tilde{\beta}_k^t$ are model coefficients. The system of equations obtained by applying the LSM to this function can be solved by the same fast method as the system of equations for function (3).

By choosing the values of the above parameters, the researcher can (within certain limits) fit the values of the unknown coefficients to the desired form. It turns out that, by applying various choice heuristics to the second summand in (4), one can not only obtain exact discrete solutions with exactly three different values of the coefficients but also construct approximate functional solutions.

When ε gradually increases, one can keep the values of the unknowns within given (though not too small) intervals by using the following restrictions in the course of optimization of function (4):

$$\begin{cases} \tilde{\gamma}_i^t = \gamma_i^t, & \text{if } |\gamma_i^t| \leq b_\gamma; & \tilde{\gamma}_i^t = -b_\gamma, & \text{if } \gamma_i^t < -b_\gamma; & \tilde{\gamma}_i^t = b_\gamma, & \text{if } \gamma_i^t > b_\gamma; \\ \tilde{\alpha}_j^t = \alpha_j^t, & \text{if } |\alpha_j^t| \leq b_\alpha; & \tilde{\alpha}_j^t = -b_\alpha, & \text{if } \alpha_j^t < -b_\alpha; & \tilde{\alpha}_j^t = b_\alpha, & \text{if } \alpha_j^t > b_\alpha; \\ \tilde{\beta}_k^t = \beta_k^t, & \text{if } |\beta_k^t| \leq b_\beta; & \tilde{\beta}_k^t = -b_\beta, & \text{if } \beta_k^t < -b_\beta; & \tilde{\beta}_k^t = b_\beta, & \text{if } \beta_k^t > b_\beta. \end{cases} \quad (5)$$

Here, γ_i^t , α_j^t , and β_k^t are the values of unknowns obtained at the preceding step, while b_γ , b_α , and b_β are nonnegative numbers.

Since S_1 contains only products of three coefficients, it is easy to see that the minimum of objective function (4) under constraints (5) depends on the single argument $b = (b_\gamma b_\alpha b_\beta)^{\frac{1}{3}}$. It turns out that, in the search for exact discrete solutions, it is always possible to choose identical limits. Differing limits b_γ , b_α , and b_β are needed for choosing certain functional solutions.

The minimum degrees of Laurent polynomials and, accordingly, the numbers b_γ , b_α , and b_β can be chosen as follows. First, the value of q in the expression $S_1 = O(b^q)$ is experimentally estimated by minimizing objective function (4) for a sufficiently large ε . Suppose that the solution can be represented by Laurent polynomials. Then the value $h = 6/q$, called the order of the polynomial solution, is the order of the approximate decomposition of the matrix multiplication tensor introduced in [18]. The knowledge of this order, allows one to choose the minimum degrees of the Laurent polynomials for each of the three groups of coefficients. (The sum of these degrees is $-h$.) It turns out that one can seek solutions for which all the polynomials of minimum degree in each group have the form $\pm x^p$, where the exponents p are nonpositive and, in general, different for different groups. By evaluating these polynomials for a certain x in the interval $[0.05, 0.2]$, one obtains the limits b_γ , b_α , and b_β needed for the subsequent search for a polynomial solution.

The heuristic algorithm presented below makes it possible to discretize solutions to problems of exact matrix multiplication. It can also be the third step in a procedure for constructing functional solutions for problems of approximate matrix multiplication.

Assign some small value ε . Minimize (4) iteratively under the constraints (5) with $b_\gamma = 0$, $b_\alpha = 0$, and $b_\beta = 0$ until the value of objective function (4) is stably less than one.

Assign some positive numbers B_γ , B_α , B_β and numbers M_γ , M_α , M_β that are initially equal to one.

After each iteration step, arrange the current values of the unknown coefficients according to increasing moduli. Choose $\tilde{\gamma}_i^t$, $\tilde{\alpha}_j^t$, and $\tilde{\beta}_k^t$ in accordance with (5), setting $b_\gamma = B_\gamma$, $b_\alpha = B_\alpha$, $b_\beta = B_\beta$ for M_γ , M_α , and M_β coefficients, respectively, with the largest moduli and $b_\gamma = 0$, $b_\alpha = 0$, $b_\beta = 0$ for the other coefficients.

Continuing minimization (4), increase gradually the numbers M_γ , M_α , M_β , and ε so as to keep the values of the objective function sufficiently small.

The process described above often results in either an exact discrete solution or an approximate numerical solution with a large number of zero coefficients having also a large number of coefficients with an identical maximum modulus in each group. These maximum coefficients are associated with the minimum degree Laurent polynomials. The last stage in the search for a polynomial solution is described at the end of this paper.

Table 1 presents the minimum lengths of solutions obtained by this author. These lengths are upper bounds for the ranks of the problems of exact and approximate matrix multiplication. The asterisks indicate estimates for the problems of exact multiplication whose solutions we were not able to reduce to the discrete form. (In such cases, a solution for which $S_1 < 10^{-12}$ and all the coefficients do not exceed one in modulus was regarded as an exact solution.) In parentheses, we show the lengths of the calculated approximate numerical solutions for which we did not so far succeed in obtaining functional solutions.

The size of Table 1 is explained by two reasons. For large-scale problems, our method does not lead to exact solutions even in the cases where such solutions do exist. In particular, we were not able to obtain the exact solution (4, 4, 4; 49) using straightforward optimization, although we found an approximate solution of the type (4, 4, 4; 46). The problems related to approximate solutions concern the accuracy and time of calculations. This part of the table is more promising for improving and extending if a more powerful computer is available.

A number of analogous calculations conducted for complex numbers did not improve the results obtained for real numbers.

Among estimates for exact ranks, the estimates numbered 20, 21, 27, and 32 are new.

Among estimates for the boundary rank, all the estimates are new with the exceptions of the estimates numbered 1, 2, 3, and, partially, 5 (an approximate solution of the type (2, 2, 4; 13) and an approximate solution of the type (2, 2, 5; 16) for fields of characteristic different from two were presented in [19]).

For problems of the type (2, n_2 , n_3), all the solutions lengths that we obtained satisfy the formula

$$R(2, n_2, n_3) = \left\lceil \frac{(3n_2 + 1)n_3}{2} \right\rceil,$$

which is a well-known upper bound (see [20]).

For $n_2 \geq 3$, the tabulated estimates $R(n_2, n_2, n_3)$ can be compared with the theoretical value $R = 2n_3n_2 + 2n_2 - n_3 - 2$, which gives a lower bound for the bilinear complexity of multiplying rectangular matrices (see [21]). For problems of the type (3, 3, n_3), the ratio of the tabulated upper estimates to the theoretical lower bounds is close to 1.2.

In the last two equalities, the arguments are arranged in nondecreasing order.

Note that the product of two 54×54 matrices can be represented as the product of block matrices with the block sizes 3×3 and 3×6 , respectively. In turn, the blocks of the left factor can be regarded as block matrices of the block size 3×6 formed of 6×3 matrices with scalar entries, while the blocks of the right factor can be regarded as block matrices of the block size 6×3 formed of 3×3 matrices. For the composite solution (54, 54, 54; 40^3), the exponent in the asymptotic estimate of arithmetic complexity is $p = 3\log_{54}(40) = 2.7743$. Thus, the solution (3, 3, 6; 40) yields a practical algorithm for exact multiplication of square matrices that is faster than the available practical algorithms for this problem. There is no need in the explicit construction of the composite solution because the algorithm for multiplying rectangular matrices can be applied directly by partitioning the square matrices under multiplication as indicated above. In this way, greater simplicity and efficiency are achieved, as well as greater flexibility in combination with other algorithms. Below, in Table 2, we present the solution (3, 3, 6; 40).

Two other solutions corresponding to the permuted dimensions can be obtained via the corresponding permutations of groups (columns) of the coefficient matrices and their transposition. When the columns of one matrix factor and the product are permuted, the coefficient matrix of the non-permuted factor is transposed. If the columns in both factors are permuted, then all the coefficient matrices should be transposed.

There is a related solution to this problem having the values in the set $\{-1/2, 0, 1/2\}$ and the same number of zero coefficients. In contrast to other problems for which discrete solutions were found, we were not able to bring the solution (3, 3, 6; 40) to a form with the values in the set $\{-1, 0, 1\}$. Summing this solution and then the result of summation with itself in dimension three, we conclude that $R(6, 6, 6) \leq 160$.

Table 1

No.	Size of the problem			$n_1 n_2 n_3$	Estimate for the rank of exact multiplication R	Estimate for the rank of approximate multi- plication r
	n_1	n_2	n_3			
1	2	2	2	8	7	7
2	2	2	3	12	11	10
3	2	2	4	16	14	13
4	2	3	3	18	15	14
5	2	2	5	20	18	16
6	2	3	4	24	20	19
7	2	2	6	24	21	20 (19)
8	3	3	3	27	23	20
9	2	2	7	28	25	23 (22)
10	2	3	5	30	25	23
11	2	4	4	32	26	24
12	2	2	8	32	28	26
13	3	3	4	36	29	27
14	2	3	6	36	30	28
15	2	2	9	36	32	29
16	2	4	5	40	33	32 (30)
17	2	2	10	40	35	32
18	2	3	7	42	35	33 (32)
19	2	2	11	44	39	36 (35)
20	3	3	5	45	36	34
21	3	4	4	48	38	35
22	2	4	6	48	39	37 (36)
23	2	3	8	48	40	37
24	2	2	12	48	42	39 (38)
25	2	5	5	50	40	39 (38)
26	2	2	13	52	46	43 (41)
27	3	3	6	54	40	39
28	2	3	9	54	45	42
29	2	4	7	56	46	43
30	2	2	14	56	49	46 (44)
31	2	5	6	60	48*	46
32	3	4	5	60	48	46 (45)
33	2	3	10	60	50	47 (46)
34	2	2	15	60	53	49 (48)
35	3	3	7	63	49	47
36	4	4	4	64	49	46
37	2	4	8	64	52*	48
38	2	2	16	64	56	52 (51)

We succeeded in reducing the number of nonzero coefficients in the exact solution (3, 3, 3; 23). The number of nonzero coefficients in the algorithm presented in Table 3 is less than the corresponding numbers in the solutions found in [13, 14]. The coefficient matrices in the table are unfolded row-wise.

From the viewpoint of “practical programming,” the solutions presented above are very unripe. It is easy to see that the linear forms in the tables contain subsets of two or more identical nonzero coefficients

Table 2. Algorithm for the exact multiplication of 3-by-3 matrix by 3-by-6 matrix involving 40 active multiplications

t	γ^t						α^t			β^t					
1	-1/8	-1	0	0	0	1	1	0	1	0	-1/8	0	0	1	1/8
	-1/8	0	1	1	0	0	0	1	1	0	0	1/8	1/8	-1	0
	0	1	0	1	-1/8	0	-1	1	0	-1	0	1/8	0	0	1/8
2	1/8	0	-1	-1	0	0	1	0	-1	0	0	-1/8	-1/8	-1	0
	1/8	1	0	0	0	-1	0	1	-1	0	1/8	0	0	1	-1/8
	0	0	1	0	1/8	-1	-1	1	0	-1	-1/8	0	1/8	0	0
3	0	0	-1	0	-1/8	1	-1	0	-1	-1	0	1/8	0	0	-1/8
	0	1	0	1	-1/8	0	0	1	-1	-1	1/8	0	1/8	0	0
	-1/8	0	1	1	0	0	1	1	0	0	-1/8	0	0	1	-1/8
4	-1/8	1	0	0	0	1	-1	0	1	0	-1/8	0	0	-1	-1/8
	1/8	0	1	-1	0	0	0	-1	-1	0	0	-1/8	1/8	-1	0
	0	-1	0	1	-1/8	0	1	1	0	-1	0	-1/8	0	0	1/8
5	-1/8	0	-1	1	0	0	1	0	1	0	0	-1/8	1/8	1	0
	1/8	-1	0	0	0	-1	0	1	-1	0	-1/8	0	0	1	-1/8
	0	0	1	0	-1/8	1	-1	-1	0	-1	1/8	0	1/8	0	0
6	0	1	0	-1	-1/8	0	-1	0	1	-1	-1/8	0	1/8	0	0
	0	0	1	0	1/8	1	0	1	-1	1	0	1/8	0	0	1/8
	1/8	1	0	0	0	1	-1	1	0	0	0	-1/8	-1/8	-1	0
7	0	0	-1	0	-1/8	-1	-1	0	-1	-1	0	1/8	0	0	1/8
	0	-1	0	1	1/8	0	0	1	1	1	-1/8	0	1/8	0	0
	1/8	0	-1	1	0	0	-1	1	0	0	-1/8	0	0	1	1/8
8	1/8	1	0	0	0	1	1	0	1	0	1/8	0	0	-1	1/8
	-1/8	0	1	-1	0	0	0	1	-1	0	0	1/8	-1/8	-1	0
	0	1	0	-1	-1/8	0	1	1	0	1	0	-1/8	0	0	1/8
9	0	0	1	0	-1/8	1	-1	0	1	-1	0	-1/8	0	0	-1/8
	0	1	0	-1	1/8	0	0	1	-1	1	1/8	0	-1/8	0	0
	-1/8	0	-1	1	0	0	1	-1	0	0	-1/8	0	0	-1	1/8
10	0	-1	0	1	-1/8	0	-1	0	-1	-1	1/8	0	-1/8	0	0
	0	0	-1	0	1/8	-1	0	1	1	1	0	-1/8	0	0	-1/8
	-1/8	1	0	0	0	1	1	-1	0	0	0	-1/8	-1/8	1	0
11	0	1	0	1	1/8	0	1	0	1	-1	1/8	0	1/8	0	0
	0	0	-1	0	1/8	1	0	-1	1	-1	0	1/8	0	0	-1/8
	-1/8	1	0	0	0	-1	1	1	0	0	0	-1/8	1/8	1	0
12	-1/8	0	-1	-1	0	0	1	0	1	0	0	-1/8	-1/8	1	0
	-1/8	1	0	0	0	-1	0	1	1	0	1/8	0	0	-1	-1/8
	0	0	-1	0	1/8	1	1	-1	0	-1	1/8	0	-1/8	0	0
13	-1/8	1	0	0	0	-1	-1	0	1	0	-1/8	0	0	-1	1/8
	-1/8	0	-1	-1	0	0	0	-1	1	0	0	1/8	1/8	1	0
	0	1	0	1	1/8	0	-1	1	0	-1	0	-1/8	0	0	-1/8
14	0	0	-1	0	1/8	1	-1	0	1	1	0	1/8	0	0	-1/8
	0	1	0	1	1/8	0	0	1	1	1	1/8	0	1/8	0	0
	-1/8	0	-1	-1	0	0	-1	-1	0	0	1/8	0	0	1	1/8

Table 2. (Contd.)

t	γ^t						α^t			β^t					
15	0	-1	0	-1	1/8	0	-1	0	1	1	1/8	0	1/8	0	0
	0	0	1	0	1/8	-1	0	1	1	1	0	1/8	0	0	-1/8
	1/8	1	0	0	0	-1	1	1	0	0	0	1/8	-1/8	1	0
16	-1/8	0	1	-1	0	0	1	0	-1	0	0	1/8	-1/8	1	0
	1/8	1	0	0	0	1	0	1	1	0	1/8	0	0	1	1/8
	0	0	1	0	1/8	1	1	1	0	1	1/8	0	1/8	0	0
17	-1/8	0	0	0	-1/8	0	0	0	0	-1	-1/8	1/8	-1/8	1	-1/8
	1/8	1	-1	1	-1/8	1	-1	-1	0	-1	0	0	0	1	0
	0	0	0	1	0	1	-1	-1	0	0	0	0	1/8	0	-1/8
18	-1/8	1	-1	1	-1/8	1	1	1	0	-1	0	0	0	-1	0
	1/8	0	0	0	-1/8	0	0	0	0	-1	1/8	-1/8	1/8	-1	1/8
	0	-1	1	0	0	0	-1	-1	0	0	-1/8	-1/8	0	0	0
19	1/8	0	0	0	1/8	0	0	0	0	-1	-1/8	1/8	1/8	1	1/8
	1/8	1	-1	-1	-1/8	-1	-1	1	0	1	0	0	0	-1	0
	0	0	0	-1	0	-1	-1	1	0	0	0	0	-1/8	0	1/8
20	1/8	1	1	1	-1/8	-1	0	1	1	1	0	0	0	1	0
	1/8	0	0	0	-1/8	0	0	1	1	1	1/8	1/8	1/8	-1	-1/8
	0	-1	1	0	0	0	0	0	0	0	1/8	1/8	0	0	0
21	-1/8	0	0	0	-1/8	0	-1	0	1	1	1/8	1/8	-1/8	1	1/8
	1/8	1	1	-1	1/8	1	1	0	-1	-1	0	0	0	1	0
	0	0	0	1	0	1	0	0	0	0	0	0	1/8	0	-1/8
22	-1/8	0	0	0	-1/8	0	-1	0	-1	1	-1/8	-1/8	-1/8	1	1/8
	-1/8	1	1	1	-1/8	-1	-1	0	-1	1	0	0	0	-1	0
	0	0	0	1	0	1	0	0	0	0	0	0	-1/8	0	1/8
23	0	1	1	0	0	0	0	0	0	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	-1	1	0	-1/8	1/8	0	0	0
	1/8	1	-1	-1	-1/8	1	0	-1	1	1	1/8	-1/8	-1/8	-1	1/8
24	0	-1	-1	0	0	0	0	0	0	0	0	0	1/8	0	1/8
	0	0	0	1	0	-1	0	-1	-1	0	-1/8	1/8	0	0	0
	1/8	1	-1	1	-1/8	-1	0	-1	-1	-1	-1/8	1/8	-1/8	1	1/8
25	-1/8	-1	1	1	-1/8	1	-1	1	0	1	0	0	0	1	0
	-1/8	0	0	0	1/8	0	0	0	0	-1	-1/8	1/8	1/8	-1	1/8
	0	1	-1	0	0	0	1	-1	0	0	1/8	1/8	0	0	0
26	-1/8	-1	-1	1	1/8	-1	0	-1	1	-1	0	0	0	-1	0
	1/8	0	0	0	-1/8	0	0	1	-1	1	1/8	1/8	-1/8	-1	1/8
	0	-1	1	0	0	0	0	0	0	0	-1/8	-1/8	0	0	0
27	0	1	1	0	0	0	1	0	1	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	0	0	0	-1/8	1/8	0	0	0
	1/8	-1	-1	-1	1/8	-1	-1	0	-1	-1	1/8	1/8	1/8	-1	1/8
28	0	1	1	0	0	0	-1	0	-1	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	0	0	0	-1/8	1/8	0	0	0
	-1/8	1	1	-1	-1/8	-1	-1	0	-1	1	-1/8	-1/8	1/8	1	1/8

Table 2. (Contd.)

t	γ^t						α^t			β^t					
29	0	1	1	0	0	0	1	0	-1	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	0	0	0	-1/8	1/8	0	0	0
	1/8	1	1	1	1/8	1	1	0	-1	-1	-1/8	-1/8	-1/8	-1	-1/8
30	0	1	1	0	0	0	0	0	0	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	1	-1	0	-1/8	1/8	0	0	0
	-1/8	1	-1	1	1/8	-1	0	-1	1	-1	1/8	-1/8	1/8	1	-1/8
31	0	1	1	0	0	0	0	0	0	0	0	0	1/8	0	1/8
	0	0	0	-1	0	1	0	-1	-1	0	-1/8	1/8	0	0	0
	1/8	-1	1	1	-1/8	-1	0	1	1	1	-1/8	1/8	1/8	-1	-1/8
32	0	-1	-1	0	0	0	1	0	-1	0	0	0	1/8	0	1/8
	0	0	0	1	0	-1	0	0	0	0	-1/8	1/8	0	0	0
	1/8	1	1	-1	1/8	-1	-1	0	1	1	1/8	1/8	-1/8	1	-1/8
33	-1/8	1	-1	-1	-1/8	-1	1	-1	0	-1	0	0	0	-1	0
	-1/8	0	0	0	1/8	0	0	0	0	1	-1/8	1/8	1/8	1	1/8
	0	1	-1	0	0	0	1	-1	0	0	-1/8	-1/8	0	0	0
34	1/8	0	0	0	1/8	0	1	0	-1	1	1/8	1/8	1/8	1	-1/8
	1/8	1	1	1	1/8	-1	1	0	-1	1	0	0	0	-1	0
	0	0	0	-1	0	-1	0	0	0	0	0	0	-1/8	0	1/8
35	-1/8	1	1	1	1/8	-1	0	1	-1	-1	0	0	0	-1	0
	-1/8	0	0	0	1/8	0	0	1	-1	-1	1/8	1/8	1/8	1	-1/8
	0	1	-1	0	0	0	0	0	0	0	-1/8	-1/8	0	0	0
36	-1/8	-1	1	-1	-1/8	-1	1	1	0	-1	0	0	0	-1	0
	1/8	0	0	0	-1/8	0	0	0	0	-1	-1/8	1/8	-1/8	-1	-1/8
	0	-1	1	0	0	0	1	1	0	0	-1/8	-1/8	0	0	0
37	1/8	0	0	0	1/8	0	0	0	0	-1	1/8	-1/8	-1/8	1	-1/8
	1/8	-1	1	1	-1/8	1	-1	1	0	1	0	0	0	-1	0
	0	0	0	-1	0	-1	1	-1	0	0	0	0	-1/8	0	1/8
38	1/8	0	0	0	1/8	0	0	0	0	1	-1/8	1/8	-1/8	-1	-1/8
	-1/8	1	-1	1	1/8	1	-1	-1	0	1	0	0	0	-1	0
	0	0	0	-1	0	-1	1	1	0	0	0	0	-1/8	0	1/8
39	1/8	-1	-1	1	-1/8	-1	0	-1	-1	1	0	0	0	1	0
	-1/8	0	0	0	1/8	0	0	1	1	-1	1/8	1/8	-1/8	1	1/8
	0	1	-1	0	0	0	0	0	0	0	1/8	1/8	0	0	0
40	1/8	0	0	0	1/8	0	-1	0	-1	-1	1/8	1/8	-1/8	-1	1/8
	-1/8	1	1	-1	-1/8	1	1	0	1	1	0	0	0	-1	0
	0	0	0	-1	0	-1	0	0	0	0	0	0	-1/8	0	1/8

inside each of the groups. Thus, an optimization of calculations is required in order to reduce the constant in the ultimate algorithm. In addition, the stability of the algorithms should be examined via the methodology proposed in [22].

For the solution (3, 3, 6; 40), the trivial optimization by means of eliminating duplicate calculations reduced the number of inactive operations from 1246 to 353 (including divisions by 8). For the solution (3, 3, 3; 23), the corresponding reduction was from 84 to 64. Our preliminary estimates allow us to conjecture that the most rapid practical algorithm should be a composite algorithm that uses different solu-

Table 3. Algorithm for the exact multiplication of 3-by-3 matrices involving 23 active multiplications and 139 nonzero coefficients

t	γ^t (45 nonzeros)	α^t (49 nonzeros)	β^t (45 nonzeros)
1	-1 0 0 1 0 0 1 0 0	0 0 0 0 0 0 0 0 1	-1 0 0 -1 0 0 1 0 0
2	0 0 0 -1 0 0 0 1 0	0 0 0 0 1 0 0 0 1	0 0 0 -1 0 0 0 1 0
3	0 -1 1 0 0 0 0 0 0	1 1 0 0 0 0 1 1 0	0 0 0 0 0 1 0 0 0
4	0 0 0 0 0 1 0 0 0	-1 0 0 1 0 0 0 0 0	0 1 1 0 0 0 0 0 0
5	0 -1 1 0 0 0 0 1 -1	0 0 0 0 0 0 -1 -1 0	0 -1 0 0 0 0 0 0 0
6	0 1 0 1 1 0 0 0 0	0 0 0 0 1 1 0 0 0	0 0 0 0 0 0 0 1 0
7	0 0 0 -1 0 0 0 0 0	0 0 0 -1 0 0 0 0 -1	1 0 0 0 0 0 0 0 0
8	0 0 0 0 0 0 1 0 -1	0 0 0 0 0 0 1 0 1	1 0 0 0 0 0 0 0 0
9	0 0 0 0 1 -1 0 1 0	0 0 0 0 1 0 0 0 0	0 -1 0 0 1 0 0 -1 0
10	0 0 0 0 0 0 0 1 0	0 0 0 0 1 0 0 -1 0	0 1 0 1 -1 0 0 0 0
11	0 0 0 0 0 0 -1 -1 0	0 0 0 0 0 0 0 -1 -1	0 0 0 1 0 0 0 0 0
12	0 1 0 0 0 1 0 0 0	0 0 -1 0 1 1 0 0 0	0 0 0 0 1 1 0 -1 0
13	1 0 0 0 0 0 0 0 0	0 0 1 0 0 0 0 0 1	0 0 0 0 0 0 1 0 1
14	0 0 0 0 0 0 0 0 1	1 0 0 0 0 0 1 0 0	1 0 1 0 0 -1 0 0 0
15	-1 0 0 0 0 0 0 0 1	-1 0 0 0 0 0 0 0 1	1 0 0 0 0 0 0 0 1
16	0 0 0 0 0 1 0 0 0	0 0 -1 0 0 1 0 0 0	0 0 0 0 -1 -1 0 1 1
17	0 1 0 0 0 0 0 0 0	0 -1 -1 0 1 1 0 0 0	0 0 0 0 -1 -1 0 0 0
18	0 0 0 1 0 0 0 0 0	0 0 0 0 0 -1 0 0 1	0 0 0 0 0 0 -1 1 0
19	0 1 -1 0 0 0 0 0 1	-1 0 0 0 0 0 -1 -1 0	0 -1 0 0 0 -1 0 0 0
20	1 0 -1 0 0 -1 0 0 0	-1 0 -1 0 0 0 0 0 0	0 0 0 0 0 0 0 0 1
21	0 0 -1 0 0 -1 0 0 1	1 0 0 0 0 0 0 0 0	0 -1 -1 0 0 0 0 0 1
22	0 0 0 0 -1 1 0 0 0	0 0 0 -1 -1 0 0 0 0	0 1 0 0 0 0 0 0 0
23	-1 0 0 0 0 0 0 0 0	0 -1 0 0 0 0 0 0 1	0 0 0 1 0 0 0 0 0

tions in the process of decomposing and reducing the dimensions of matrix factors, for instance, $(3, 3, 6; 40)$, $(2, 2, 2; 7)$, $(3, 3, 3; 23)$, and $(n_1, n_2, n_3; n_1 n_2 n_3)$. For the reasonable range of matrix sizes, such a scheme with an optimal choice of the number of steps for each solution involved should be faster than the existing algorithms. It is important that, in the case of an asymmetric solution, say, $(3, 3, 6; 40)$, not all the three steps need to be used for symmetrization. Symmetrization can be performed at the final stage via the ordinary multiplication of rectangular matrices. It may be more advantageous to use other low-dimensional asymmetric solutions just before the final stage.

Now, we turn to algorithms for approximate matrix multiplication.

From line 8 in Table 1, we see that $r(3, 3, 3) \leq 20$.

Line 36 in Table 1 shows that $r(4, 4, 4) \leq 46$. The functional solution $(4, 4, 4; 46)$ is an ordered collection of 2208 functions of which 467 functions do not identically vanish. A non-computer analysis of such data is difficult. To avoid errors, the author prefers not to present this algorithm (and some other algorithms) in this paper. However, everyone interested in these algorithms can obtain them from the author in the electronic form.

Lines 4, 8, 13, and 21 in Table 1 imply that there exist several algorithms for approximate matrix multiplication that are asymptotically faster than the well-known algorithm $(3, 3, 3; 21)$ proposed in [6]. Algorithms for multiplying square matrices can be derived from those for rectangular matrices via the symmetrization method described above.

The solution $(2, 2, 4; 13)$ over an arbitrary field and the solution $(2, 2, 5; 16)$ over an arbitrary field of characteristic different from two have already been published. Therefore, in Tables 4–6, we present

Table 4. Algorithm for the approximate multiplication of 3-by-2 matrix by 2-by-3 matrix involving 14 active multiplications

t	γ^t			α^t		β^t		
1	0	-1	1	0	x^{-1}	0	x^{-1}	0
	-1	1	$-x^{-1}$	x	0	0	0	0
	x	$-x^{-1}$	x^{-1}	0	0			
2	x	0	$-x^2$	0	x^{-1}	x^{-1}	0	0
	0	0	x^{-1}	0	0	0	0	0
	x^{-1}	-1	$-x^{-1}$	x^2	-1			
3	$-x^2$	0	x^2	$-x^{-1}$	0	x^{-1}	0	x
	0	0	0	0	0	0	0	$-x$
	0	0	x^{-1}	$1 + x^2$	0			
4	0	0	0	0	0	1	1	x
	1	0	x^{-1}	1	0	0	0	0
	$-x$	-1	0	0	0			
5	0	0	x^2	$-x^{-1}$	$-x^{-1}$	$-x^{-1}$	0	0
	0	0	0	0	0	0	0	x
	0	0	x^{-1}	1	1			
6	0	0	0	0	$-x^{-1}$	$-x^{-1}$	0	0
	0	0	$-x^{-1}$	$-x$	0	0	1	0
	0	1	0	$-x^2$	1			
7	x	0	0	0	$-x^{-1}$	x^{-1}	0	0
	0	0	0	x^3	0	x	0	0
	x^{-1}	0	0	0	1			
8	0	0	0	0	$-x^{-1}$	0	$-x^{-1}$	0
	1	-1	x^{-1}	0	0	x^{-1}	0	0
	$-x$	0	0	0	0			
9	0	0	0	0	x^{-1}	0	0	0
	0	0	$-x^{-1}$	x	-1	0	1	0
	0	0	0	x^2	-1			
10	0	x^2	1	x^{-1}	0	0	x^{-1}	x
	0	0	0	0	0	0	0	$-x$
	0	0	x^{-1}	0	0			
11	0	0	1	x^{-1}	x^{-1}	0	$-x^{-1}$	0
	0	0	0	x	0	0	0	x
	0	0	x^{-1}	0	0			
12	$x^2 + x^3$	0	0	0	x^{-1}	0	0	0
	1	-1	x^{-1}	0	x	x^{-1}	0	0
	0	0	0	0	0			
13	0	1	0	0	x^{-1}	0	x^{-1}	0
	0	0	0	$x + x^2$	0	0	x	0
	0	x^{-1}	0	x^2	0			
14	0	0	0	0	0	0	0	0
	0	-1	x^{-1}	0	1	-1	-1	x
	0	0	0	0	0			

Table 5. Algorithm for the approximate multiplication of 2-by-2 matrix by 2-by-3 matrix with the zero entry (2, 3). The algorithm involves 8 active multiplications

t	γ^t			α^t		β^t		
1	$1-x$	x^{-1}	0	1	x	1	0	$-x^{-1}$
	x	0	$-x^2$	x^{-1}	0	0	0	0
2	1	x^{-1}	0	0	x	-1	$-x^2$	x^{-1}
	0	0	0	x^{-1}	0	x^{-1}	0	0
3	0	x^{-1}	0	x^{-1}	0	$-x$	0	1
	0	$-x^{-1}$	0	x^{-1}	0	x^{-1}	-1	0
4	0	0	0	0	0	$x+x^2$	x^2	0
	1	x^{-1}	0	x^{-1}	$-x$	$-x^{-1}$	0	0
5	-1	$-x^{-1}$	0	0	0	$-x$	0	1
	1	x^{-1}	0	x^{-1}	0	x^{-1}	0	0
6	$1-x$	0	x	x^{-1}	0	x^2	0	1
	x	x^{-1}	$-x^2$	0	0	0	0	0
7	0	0	0	$-x^{-1}$	0	$x+x^2$	0	0
	0	x^{-1}	0	$-x^{-1}$	x	$-x^{-1}$	1	0
8	0	x^{-1}	0	x^{-1}	x	0	x^2	0
	0	0	0	x^{-1}	0	$-x^{-1}$	1	0

(in functional form) the solutions (2, 3, 3; 14), (2, 2, 5; 16), and (3, 3, 3; 20) that are valid over an arbitrary field.

For the solution in Table 4,

$$S_1 = 7x^2 + O(x^3) = O(b^{-2}).$$

This solution is of polynomial order three.

In Table 5, we show the first half of the solution (2, 2, 5; 16). This is the solution (2, 2, 3; 8) for the partial multiplication problem with the zero entry a_{23} . The second half can be obtained from the presented solution by simultaneous permutations of matrix rows (columns) that are admitted in (1) and correspond to moving the entry a_{23} into position (1, 2); then a shift of the matrices α^t and β^t is performed. For solution in the table,

$$S_1 = 7x^2 + O(x^3) = O(b^{-2}).$$

This solution is of polynomial order three. For fields of characteristic different from two, there exists a solution of the second order.

For the solution in Table 6,

$$S_1 = 14x^2 + O(x^3) = O(b^{-1}).$$

This solution is of polynomial order six.

We obtained most of the functional algorithms from numerical solutions to problems of approximate matrix multiplication. The heuristic fitting of the numerical coefficients to the values of Laurent polynomials of the argument $x = 0.1$ was used. Only the integers with the absolute value less than five and, in certain cases, the fraction $1/2$ were admitted as the coefficients of polynomials. This restriction ensures one-to-one correspondence between the polynomials and numerical coefficients. The fitting in the course of optimization was performed in accordance with formula (4). The numerical coefficients representing the polynomials were separated into two categories, namely, free coefficients with the absolute values restricted by relations (5) and coefficients $\tilde{\gamma}_i^t$, $\tilde{\alpha}_j^t$, and $\tilde{\beta}_k^t$ fixed at certain specific values. We examined how

Table 6. Algorithm for the approximate multiplication of 3-by-3 matrices involving 20 active multiplications

t	γ^t			α^t			β^t		
1	0	x^{-2}	0	x	0	$-x^3$	1	x	0
	$-x^2$	0	0	0	0	0	x^{-1}	0	0
	x^4	0	0	x^{-1}	0	x^{-2}	x^{-2}	0	$2x^3$
2	$-x$	0	x^{-2}	0	0	x^3	0	0	0
	$-x$	1	x	x^{-2}	$-x^{-1}$	0	0	0	x^3
	0	0	-1	0	0	$-x$	0	0	x^{-1}
3	x	0	$-x^{-2}$	0	0	0	0	0	0
	$x - x^4$	0	0	$-x^{-1}$	1	0	x^{-1}	1	$-x^2$
	0	0	1	0	$-x^{-2}$	0	0	0	0
4	$-x^{-1}$	x^{-2}	0	-1	x	0	x	0	0
	0	0	0	0	0	0	1	0	0
	x^4	0	0	$-x^{-2}$	x^{-1}	0	x^{-1}	0	0
5	0	$-x^{-2}$	0	0	0	$-x^3$	1	x	0
	x^2	0	0	x^{-2}	0	0	x^{-1}	0	0
	0	0	0	0	0	x^{-2}	x^{-2}	x^{-1}	x^3
6	1	-1	$-x^{-2}$	0	0	0	0	0	x^2
	0	0	0	0	0	0	x^{-2}	0	0
	$-x^2 - x^3$	x^2	1	x^{-2}	$-x^{-1}$	0	0	0	0
7	$-x^{-1}$	0	0	x	$-2x^2$	0	x	0	x^3
	0	0	0	0	0	0	x^{-1}	0	0
	x	0	0	x^{-2}	$-x^{-1}$	0	0	0	0
8	0	0	x^{-2}	x	0	$-x^3$	0	0	x
	$-x + x^2$	0	0	$-x^{-2}$	0	1	0	0	0
	0	0	0	x^{-1}	0	0	x^{-2}	0	0
9	$-x$	0	x^{-2}	0	0	0	0	0	0
	$-x$	$1 - x^3$	0	$-x^{-2}$	x^{-1}	0	0	x	0
	0	0	-1	0	0	0	0	0	x^{-1}
10	x	-1	$-x^{-2}$	0	0	0	0	0	x^3
	x	0	0	0	0	0	x^{-1}	1	0
	0	x^2	1	0	x^{-2}	0	0	0	0
11	0	0	x^{-2}	0	0	0	0	0	x
	$-x$	0	x	x^{-2}	0	-1	0	0	1
	0	0	0	0	0	0	0	1	0
12	0	0	$-x^{-2}$	0	x^2	0	0	0	0
	x	0	0	$-x^{-2}$	0	1	0	0	-1
	0	0	0	0	1	0	x^{-2}	0	0
13	$-x$	0	x^{-2}	0	0	0	0	0	0
	x^4	0	0	-1	x	0	x^{-2}	0	0
	0	0	-1	x^{-2}	$-x^{-1}$	0	0	0	0
14	x^{-1}	0	x^{-2}	-1	x	x^2	0	0	0
	$-x$	0	0	0	0	0	0	0	0
	0	0	0	$-x^{-2}$	x^{-1}	0	x^{-1}	0	0

Table 6. (Contd.)

t	γ^t			α^t			β^t		
15	0	x^{-1}	$-x^{-2}$	0	0	0	0	0	0
	$-x^3$	-1	$-x$	0	0	0	0	0	0
	0	x^4	0	0	0	x^{-2}	0	x^{-2}	$-x^2$
16	0	x^{-2}	0	0	$-x^2$	0	1	0	0
	0	0	0	x^{-2}	0	0	x^{-1}	-1	0
	0	0	0	0	-1	0	x^{-2}	0	0
17	0	x^{-2}	0	0	0	0	0	$-x$	x^3
	0	0	0	0	0	0	x^{-1}	0	0
	x	-1	0	x^{-1}	0	0	0	0	$-2x^3$
18	0	0	0	0	0	0	0	0	0
	0	x^2	1	$-x^{-1}$	1	x	0	0	1
	0	0	0	0	0	0	0	0	x^{-1}
19	0	0	x^{-2}	0	0	0	0	0	0
	0	1	x	-1	0	x^2	0	0	0
	0	0	0	0	0	x^{-2}	0	x^{-2}	0
20	0	x^{-2}	0	0	0	0	0	x	0
	0	x	0	x^{-2}	0	0	0	1	0
	0	0	0	0	0	0	0	x^{-1}	x^3

the choice of a single coefficient and the fixing of it at some value affect the value of the objective function. By reducing ε in (4), we can rapidly optimize the values of free coefficients, while, by increasing ε , we can test the rationality of our choice, provided that the previously chosen coefficients are kept strictly fixed. If, on completing the choice, the value of the objective function is still sufficiently small, the resulting polynomial objective function begins from a quadratic term.

By periodically recalculating the already found polynomials and, consequently, the limits for various values of x , one can obtain a more refined method for the direct choice of polynomials. Such a refinement is needed in the case where the solution contains a large number of binomials.

REFERENCES

1. V. Strassen, "Gaussian elimination is not optimal," *Numer. Math.* **13**, 354–356 (1969).
2. D. Coppersmith and S. Winograd, "Matrix multiplication via arithmetic progressions," *J. Symbolic Comput.* **9**, 251–280 (1990).
3. S. Hunold, T. Rauber, and G. Runger, "Combining building blocks for parallel multi-level matrix multiplication," *Parallel Comput.* **34**, 411–426 (2008).
4. I. Kaporin, "The aggregation and cancellation techniques as a practical tool for faster matrix multiplication," *Theor. Comput. Sci.* **315**, 469–510 (2004).
5. J. Laderman, V. Y. Pan, and X.-H. Sha, "On practical algorithms for accelerated matrix multiplication," *Linear Algebra Appl.* **162–164**, 557–588 (1992).
6. A. Schönhage, "Partial and total matrix multiplication," *SIAM J. Comput.* **10**, 434–455 (1981).
7. S. K. Sen, "Open problems in computational linear algebra," *Nonlinear Anal.* **63**, 926–934 (2005).
8. "Complexity of matrix multiplication: An overview," *Kibern. Sb.* **25**, 139–236 (1988).

9. J. M. Landsberg, "Geometry and the complexity of matrix multiplication," *Bull. Am. Math. Soc.* **45**, 247–284 (2008).
10. R. P. Brent, "Algorithms for matrix multiplications," *Comput. Sci. Dept. Report CS 157* (Stanford Univ., 1970).
11. S. Winograd, "On multiplication of 2×2 matrices," *Linear Algebra Appl.* **4**, 381–388 (1971).
12. V. B. Alekseyev, "On the complexity of some algorithms of matrix multiplication," *J. Algorithms* **6** (1), 71–85 (1985).
13. J. Laderman, "A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications," *Bull. Am. Math. Soc.* **82** (1), 126–128 (1976).
14. R. W. Johnson and A. M. McLoughlin, "Noncommutative bilinear algorithms for 3×3 matrix multiplication," *SIAM J. Comput.* **15**, 595–603 (1986).
15. O. M. Makarov, "Nekommutativnyi algoritm umnozheniya kvadratnykh matrits pyatogo poryadka, ispol'zuyushchii sto umnozhenii," *Comput. Math. Math. Phys.* **27** (2), 311–315 (1987).
16. D. Bini, "Relations between exact and approximate bilinear algorithms: Applications," *Calcolo* **17**, 87–97 (1980).
17. N. S. Bakhvalov, *Numerical Methods: Analysis, Algebra, Ordinary Differential Equations* (Nauka, Moscow, 1975; Mir, Moscow, 1977).
18. D. Bini, M. Capovani, G. Lotti, and F. Romani, " $O(n^{2.7799})$ complexity for approximate matrix multiplication," *Inf. Process. Lett.* **8** (5), 234–235 (1979).
19. V. B. Alekseev and A. V. Smirnov, "On the exact and approximate bilinear complexities of multiplication of 4×2 and 2×2 matrices," in *Mathematics and Informatics 2, Dedicated to 75th Anniversary of Anatolii Alekseevich Karatsuba, Modern Problems in Mathematics* (MIAN, Moscow, 2013), Vol. 17, pp. 135–152 [in Russian].
20. J. E. Hopcroft and L. R. Kerr, "On minimizing the number of multiplications necessary for matrix multiplication," *SIAM J. Appl. Math.* **20**, 30–36 (1971).
21. M. Bläser, "On the Complexity of the Multiplication of Matrices of Small Formats," *J. Complexity* **19** (1), 43–60 (2003).
22. D. Bini and G. Lotti, "Stability of fast algorithms for matrix multiplication," *Numer. Math.* **36**, 63–72 (1980).

Translated by Kh. Ikramov